Name: Carlos Chipantiza Neptun code: EACN90
Financial Processes (BMETE95MM14), BME, Fall 2026 course id: 54932fw

Homework 2: Implement calibrator for Tree. Part 2: Task

Remark. As in part 1, the results used are stated in boxes and cited by number with the displayed equation; no new result is needed: everything comes from the lecture, Homework 1 and Lemmas 1–2 of part 1 (sources in the References at the end). The code shown is an exact copy of the files, can be copied from this PDF into Python, and every number is printed by it (files `*.out`; the extra columns of Tables 1 and 2 in `outputs.txt`, printed by `run_all.py`). Files: `calibrator.py` (new), `bg_functions.py` (part 1, one short change), `prof_tree.py` (lecture), `hw1_call.py`, `plotter.py` (Homework 1), the runs `q1_calibrate_test.py`, `demo_A.py`, `demo_B.py`, the pictures `tree_plot.py`, `trees_demo.py`, and the notebooks `2_HW_EXE_2` (Q1), `2_HW_EXE_3` (Q2), `2_HW_EXE_4` (Q3), `2_HW_EXE_5` (Q4 trees) with the same code.

The assignment.

Task: Write a calibrate method which, for a given number of steps parameter, computes the up step size for which a Balanced Binomial Tree model returns the specified price for a given European call. (In practice option prices can be observed in the market, so it makes sense to assume it is given). Demonstrate the calibrate method for various input parameters (same contract with a few different number of steps (dont forget to scale the discount factor to keep interest rates flat); calibrate one step tree for various contracts (different expiries and strike levels, flat interest rate)). Make simple plots to illustrate results.
Hint: To do it you will need to use a root solver, I recommend using scipy.optimize.minimize.

What will be done (plan).

Notation: $V(u)$ is the call price of the balanced tree with up step u, all other parameters fixed (`balanced_tree_call_price` of part 1). The given ("market") price V^* is the Black–Scholes price of Homework 1 with $\sigma = 0.2$ and the flat rate of the tree, $r = -\ln 0.95$. Running example: $S_0 = K = 1$, $T = 1$, $DF_1 = 0.95$, $V^* = 0.105195$.

- **Q1.** Write `calibrate`: from V^*, S_0, K, T, N, DF_1 return the u with $V(u) = V^*$, in five steps (Figure 2); test it.
- **Q2 (Demo A).** Same contract, $N = 1, 2, 3, 5, 10, 20, 50, 100$ steps, discount factor scaled (Definition 3).
- **Q3 (Demo B).** One-step tree, $T \in \{0.25, 0.5, 1, 2\}$, $K \in \{0.8, 0.9, 1.0, 1.1, 1.2\}$, flat rate.
- **Q4 (plots).** Inside Q1–Q3: $V(u)$ against u; u and tree volatility against N and against K; and pictures of the calibrated trees (spot tree and price tree of the lecture code) for $N = 2, 5, 50$.

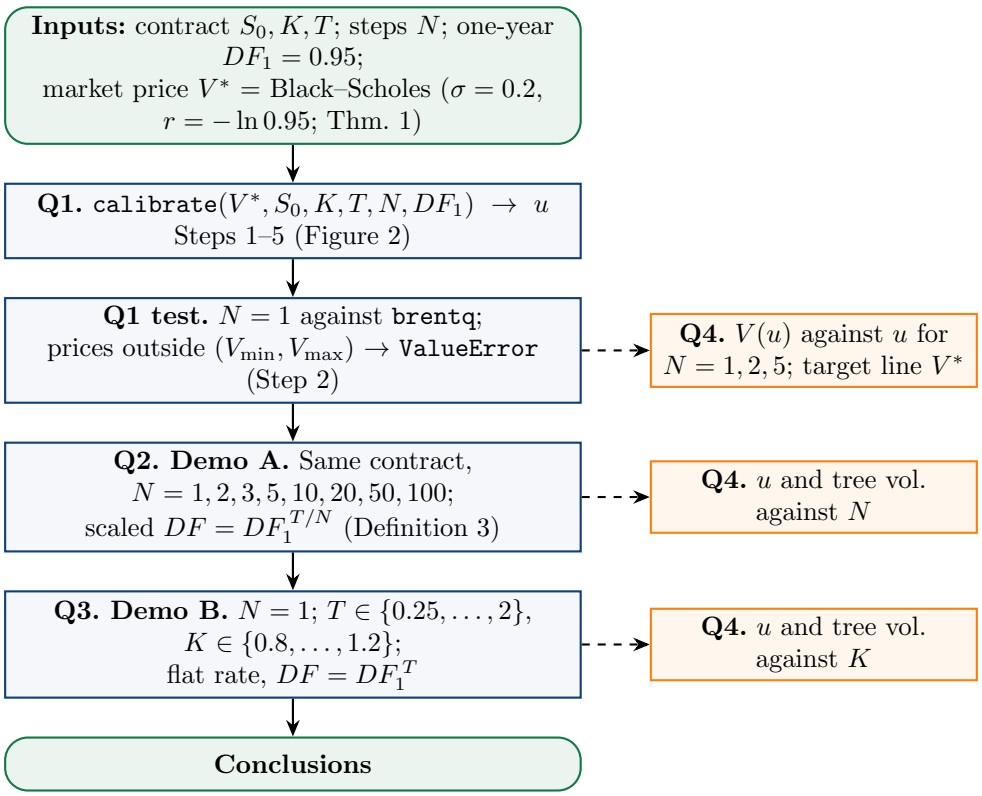


Figure 1: Master flowchart: what is done in this part.

Results used.

Definitions 1–3, Theorem 1 and Lemmas 1–2 are stated and proved (or quoted) in part 1; here they are restated briefly.

Definition 1 (Binomial tree and risk-neutral price; `Tree.ipynb`, cells 17, 19, 33, 37–38).

In one step $S_0 \to uS_0$ or dS_0, and money grows by $1 + r$, with $d < 1 + r < u$; the risk-neutral probabilities are $\tilde{p} = \frac{1+r-d}{u-d}$, $\tilde{q} = 1 - \tilde{p}$. With N steps the price of the payoff V_N is

$$V_0 = \frac{1}{(1+r)^N} \tilde{E}(V_N). \tag{1}$$

In the code (`Binomial_tree_coding.ipynb`, cell 8) the parameter is the one-step discount factor $DF = 1/(1 + r)$. Book: Shreve [3], Chapter 1 (one-period and multiperiod binomial model, Theorem 1.2.2).

Definition 2 (Balanced tree; `Tree.ipynb`, cell 56; `Binomial_tree_coding.ipynb`, cells 12–14).

A binomial tree is *balanced* if

$$\tilde{p} = \tilde{q} = \tfrac{1}{2}. \tag{2}$$

Definition 3 (Flat interest rate; `Binomial_tree_coding.ipynb`, cell 20).

For maturity T and N steps, $\Delta t = T/N$ and the one-step discount factor is

$$DF = DF_1^{\Delta t}, \tag{3}$$

so $DF^N = DF_1^T$ for every N ("scale the discount factor to keep interest rates flat").

Theorem 1 (Vega of the Black–Scholes call; Homework 1, HW 2/2 (Lesson 3, cell 49)).
The Black–Scholes call price c is strictly increasing in σ:

$$\mathcal{V} = \frac{\partial c}{\partial \sigma} = S_0\, N'(d_1)\sqrt{T} > 0. \tag{4}$$

Homework 1 computes c with `black_scholes_eur_call` (`hw1_call.py`).

Here Theorem 1 is used only to fix the market price: $V^* = c$ with $\sigma = 0.2$ and $r = -\ln DF_1$, a price that "contains" the volatility 0.2.

Lemma 1 (Balanced down step; part 1, from Definitions 1 and 2).
The tree is balanced exactly when

$$d = \frac{2}{DF} - u, \tag{5}$$

(`calcBalancedDownStep`), and $d < 1/DF < u$, $d > 0$ hold exactly when $u \in \left(\frac{1}{DF}, \frac{2}{DF}\right)$.

How it was obtained (part 1). Put $\tilde{p} = \frac{1}{2}$ (Definition 2) in $\tilde{p}$ of Definition 1, with $1 + r = 1/DF$, and solve for d:

$$\begin{aligned}
\frac{1/DF - d}{u - d} &= \frac{1}{2} && \text{(Definitions 1 and 2)} \\
\frac{2}{DF} - 2d &= u - d && \text{(multiply by } 2(u-d) > 0\text{)} \\
d &= \frac{2}{DF} - u && \text{(collect } d\text{)}
\end{aligned}$$

This is the professor's `calcBalancedDownStep`, $u - 2(u - 1/DF)$ (`Binomial_tree_coding.ipynb`, cell 13). The interval comes from the no-arbitrage condition $d < 1 + r < u$ of the lecture (`Tree.ipynb`, cell 17) and $d > 0$ (a price stays positive). Book: Shreve [3], Chapter 1 (condition $0 < d < 1 + r < u$ and risk-neutral probabilities).

Lemma 2 (Mean and spread of one step; part 1, from Definition 1 and Lemma 1).
One step multiplies the stock by u or d with probability $\frac{1}{2}$ each: mean $\frac{1}{DF}$, standard deviation $h = u - \frac{1}{DF}$ (the *step size*). The *tree volatility* is

$$\sigma_{\text{tree}} = \frac{u - 1/DF}{\sqrt{\Delta t}} \tag{6}$$

(`tree_vol` below; it is the inverse of `up_step_from_tree_vol` of part 1).

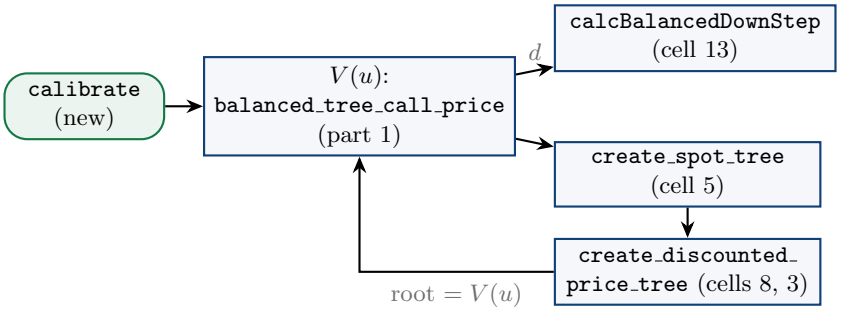
How it was obtained (part 1). With probability $\frac{1}{2}$ each (Definition 2), the mean is $\frac{1}{2}(u + d) = \frac{1}{DF}$ by Lemma 1, and both outcomes differ from the mean by $\pm\left(u - \frac{1}{DF}\right)$, so the standard deviation is $u - \frac{1}{DF}$. A mean growth of $1 + r = 1/DF$ is the "risk-neutral" property of the lecture (`Tree.ipynb`, cell 36; Shreve [3], Chapter 1). The division by $\sqrt{\Delta t}$ follows the professor's own up step in `Binomial_tree_coding.ipynb`, cell 20: $u = \exp\left((r - \sigma^2/2)\Delta t + \sigma\sqrt{\Delta t}\right)$, "step sizes are scaled as well".

Statement of the professor (assignment, Background; `Tree.ipynb`, cell 57).
"higher step is like higher volatility, it results in higher option prices. Also, there is a 1-1 correspondence between step size and price if all other parameters are fixed."
In part 1 this is (C2): $V(u)$ does not decrease when u grows; and (C3): above the flat part $V(u)$ is strictly increasing, so price and u determine each other. Checked in part 1 by figures for $N = 1, 2, 5, 50$.

The lecture code used (`Binomial_tree_coding.ipynb`).

The calibrator prices every tree with the professor's functions, unchanged (file `prof_tree.py`; the code is printed in part 1):

Cell	Function	Role in part 2
1	imports (`numpy`, `scipy.optimize.minimize`)	`minimize` is the solver of Step 4 (hint)
3	`european_call_payoff`	call payoff at the leaves
5	`create_spot_tree`	spot tree for u and d
8	`create_discounted_price_tree`	price $V(u)$ = root of the price tree (Def. 1)
13	`calcBalancedDownStep`	$d = 2/DF - u$ (Def. 2, Lemma 1)
20	(scaled discount factor)	$DF = DF_1^{T/N}$ (Def. 3; Step 1, Demo A)



Test of the lecture code (cells 6 and 14; first cells of `2_HW_EXE_2`): balanced 2-step tree, $u = 1.2$, $DF = 0.95$, $K = 1$ gives the lecture's price tree `[['0.138'], ['0.039', '0.237'], ['0.000', '0.078', '0.397']]`. Not used: the barrier cells 17–21.

Which prices can be calibrated is decided numerically in Step 2 (tree prices at the two ends of the interval of u), with the statement of the professor; no new result is needed.

Q1 — the `calibrate` method.

Idea. By the statement of the professor ((C2), (C3) of part 1), $V(u)$ does not decrease on $(1/DF, 2/DF)$; near $1/DF$ it can be constant (the flat part of Figure 6), above it is strictly increasing. So `calibrate` checks that V^* is attainable, brackets the solution above the flat part, minimizes the squared error there (the hint) and checks the result:

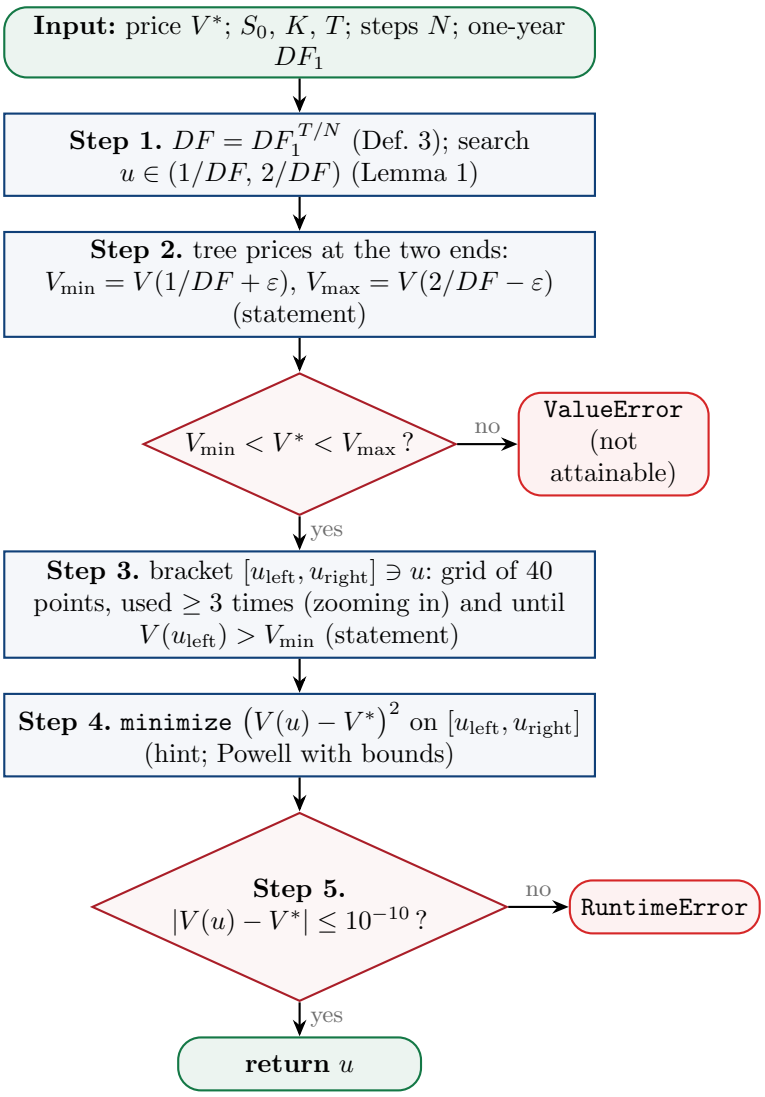


Figure 2: Flowchart of `calibrate` (file `calibrator.py`).

Step 1 (one-step DF and the interval of u). `df_annual ** (maturity / steps)` is $DF = DF_1^{T/N}$, which keeps the rate flat (Definition 3, (3)). A balanced tree needs $1/DF < u < 2/DF$ (Lemma 1, (5)): these are `u_low`, `u_high`.

Step 2 (is V^* attainable?). The function `price_range` prices the tree at the two ends of the interval: $V_{\min} = V(1/DF + \varepsilon)$, $V_{\max} = V(2/DF - \varepsilon)$, $\varepsilon = 10^{-10}/DF$. Since V does not decrease (statement), every u gives a price in $[V_{\min}, V_{\max}]$; if $V^* \notin (V_{\min}, V_{\max})$, `calibrate` raises `ValueError`.

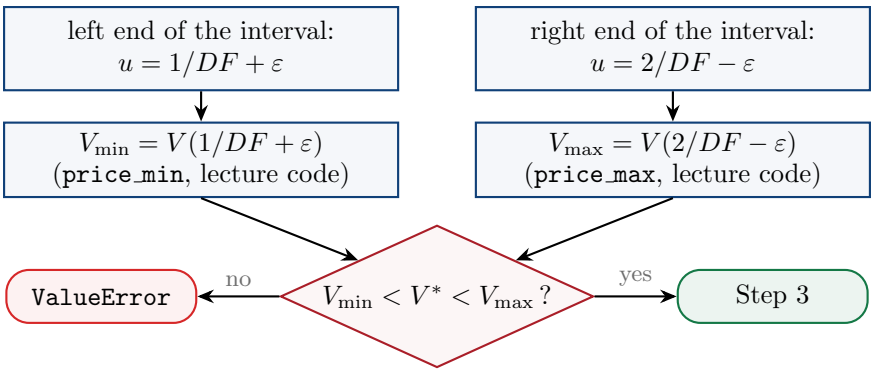


Figure 3: Step 2: the tree prices at the two ends of the interval and the test.

Step 3 (bracket). On a grid of 40 points of $(u_{\text{low}}, u_{\text{high}})$ (moved inside by $\varepsilon = 10^{-10}u_{\text{low}}$), take the first point u_k with $V(u_k) \geq V^*$. Since V does not decrease (statement), the solution lies in $[u_{k-1}, u_k]$.

- Zoom: the same grid on $[u_{k-1}, u_k]$ (39 times shorter). Grids: at least 3 (`n_zoom`: whole interval, then 2 zooms; at most 30) and until $V(u_{\text{left}}) > V_{\min}$ (code: $V_{\min} + 10^{-12}$). Then the bracket is above the flat part, where V is strictly increasing: unique solution (statement).

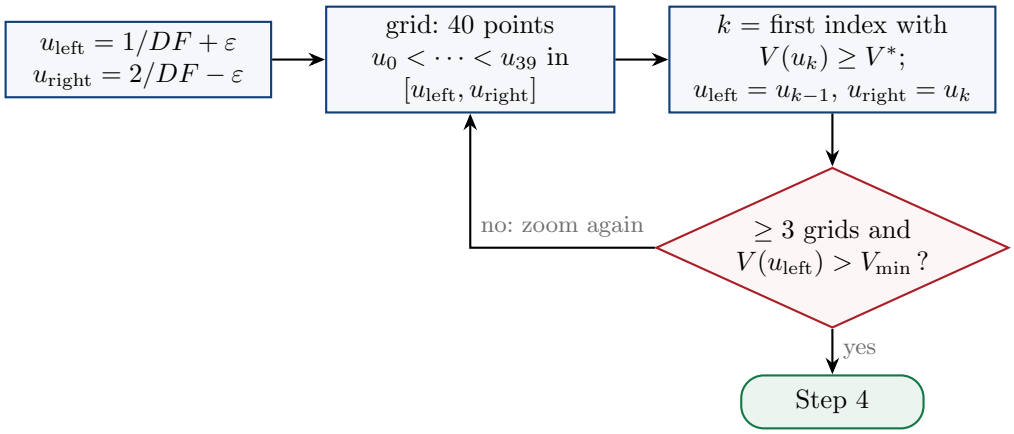


Figure 4: Step 3: bracketing by zooming in.

Step 4 (minimize). As the hint says, `scipy.optimize.minimize` is applied to $f(u) = \big(V(u) - V^*\big)^2$ on the bracket: $f \geq 0$, and $f = 0$ only at the calibrated u (unique in the bracket by Step 3). Method `"Powell"` (no derivatives, accepts bounds), start at the middle of the bracket.

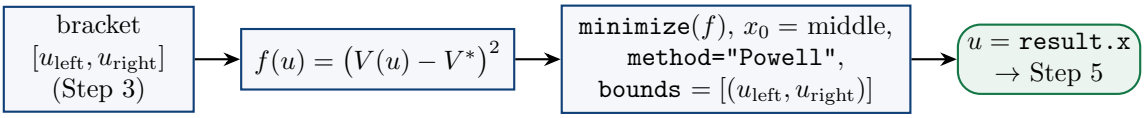


Figure 5: Step 4: the minimization of the hint.

Step 5 (check). The error $V(u) - V^*$ is computed with the tree again; if $|V(u) - V^*| > 10^{-10}$ `calibrate` raises `RuntimeError`, otherwise it returns u. So the returned u really gives "the specified price".

Q1 — the code.

The new file (Steps 1–5 are marked in the comments):

new code (`calibrator.py`)

```python
"""Homework 2, part 2 (Task): calibrator for the Balanced Binomial Tree.

calibrate(...) returns the up step u for which the balanced tree (p_up = p_down = 0.5)
prices a given European call at a given price. Steps 1-5 follow the flowchart of the report.
"""
import numpy as np
from scipy.optimize import minimize

from bg_functions import balanced_tree_call_price


def price_range(spot: float, strike: float, maturity: float, steps: int,
                df_annual: float) -> tuple[float, float]:
    """Tree prices at the two ends of the interval of u (Lemma 1): u_low + eps, u_high - eps.
    The price does not decrease in u (statement of the professor), so every u gives a price
    between these two values."""
    discount_factor = df_annual ** (maturity / steps)
    u_low, u_high = 1 / discount_factor, 2 / discount_factor
    eps = 1e-10 * u_low
    price_min = balanced_tree_call_price(spot, strike, maturity, steps, df_annual, u_low + eps)
    price_max = balanced_tree_call_price(spot, strike, maturity, steps, df_annual, u_high - eps)
    return price_min, price_max


def tree_vol(spot_mult_up: float, maturity: float, steps: int, df_annual: float) -> float:
    """Tree volatility (u - 1/DF) / sqrt(dt) of part 1 (inverse of up_step_from_tree_vol)."""
    dt = maturity / steps
    return (spot_mult_up - 1 / df_annual ** dt) / np.sqrt(dt)


def calibrate(price: float, spot: float, strike: float, maturity: float, steps: int,
              df_annual: float, n_grid: int = 40, n_zoom: int = 3,
              tol: float = 1e-10) -> float:
    """Up step u such that balanced_tree_call_price(..., u) = price."""
    # Step 1: discount factor of one step (flat rate, Definition 3) and the interval of u
    discount_factor = df_annual ** (maturity / steps)
    u_low, u_high = 1 / discount_factor, 2 / discount_factor      # d < 1/DF < u and d > 0
    # Step 2: is the price attainable? (tree prices at the two ends of the interval)
    price_min, price_max = price_range(spot, strike, maturity, steps, df_annual)
    if not price_min < price < price_max:
        raise ValueError(f"price {price:.6f} not attainable: it must lie in "
                         f"({price_min:.6f}, {price_max:.6f})")
    # Step 3: bracket the solution. The price is non-decreasing in u (C2), so on a grid of u
    #         the target lies between the last point with price < target and the first point
    #         with price >= target. Zoom in (new grid inside the bracket): use at least n_zoom
    #         grids, and stop when the left end is out of the flat part (price > price_min;
    #         near 1/DF the price can be constant).
    def tree_price(u: float) -> float:
        return balanced_tree_call_price(spot, strike, maturity, steps, df_annual, u)
    eps = 1e-10 * u_low
    u_left, u_right = u_low + eps, u_high - eps
    for zoom in range(30):
        grid = np.linspace(u_left, u_right, n_grid)
        k = next(i for i, u in enumerate(grid) if tree_price(u) >= price)
        u_left, u_right = grid[max(k - 1, 0)], grid[k]
        if zoom + 1 >= n_zoom and tree_price(u_left) > price_min + 1e-12:
            break
    # Step 4: minimize the squared pricing error inside the bracket (hint of the assignment)
    result = minimize(lambda x: (tree_price(x[0]) - price) ** 2, x0=[(u_left + u_right) / 2],
                      method="Powell", bounds=[(u_left, u_right)],
                      options={"xtol": 1e-14, "ftol": 1e-30})
    spot_mult_up = float(np.atleast_1d(result.x)[0])
    # Step 5: check the result
    error = tree_price(spot_mult_up) - price
    if abs(error) > tol:
        raise RuntimeError(f"calibration failed: pricing error {error:.2e}")
    return spot_mult_up
```

The part 1 file `bg_functions.py` had no imports (they were in the notebook); one change adds them, so that `calibrator.py` can import it:

change in bg_functions.py (top of the file; the rest is the code of part 1)

```
# change # 1  (part 2: imports added so that this file can be imported as a module)
import numpy as np
from prof_tree import create_spot_tree, create_discounted_price_tree, calcBalancedDownStep
# end change # 1
```

Q1 — test.

(a) $N = 1$ against `scipy.optimize.brentq` (another root solver); (b) two prices outside the range of Step 2; (c) the picture (Q4).

test (q1_calibrate_test.py)

```
# Q1: test of calibrate (N = 1 against another root solver, prices out of range, picture)
import numpy as np
from scipy.optimize import brentq
from plotter import my_plotter
from hw1_call import black_scholes_eur_call
from bg_functions import balanced_tree_call_price
from calibrator import calibrate, price_range

spot, strike, maturity, df_annual = 1.0, 1.0, 1.0, 0.95
r = -np.log(df_annual)                                    # the same flat rate
target = black_scholes_eur_call(r=r, T=maturity, S0=spot, sigma=0.2, K=strike)[0]
print(f"target price (Black-Scholes of Homework 1, sigma = 0.2): {target:.6f}")

# (a) one step: calibrate against scipy.optimize.brentq on V(u) - V* (another root solver)
u = calibrate(target, spot, strike, maturity, 1, df_annual)
DF = df_annual ** maturity
u_brentq = brentq(lambda x: balanced_tree_call_price(spot, strike, maturity, 1, df_annual, x)
                  - target, 1 / DF + 1e-10, 2 / DF - 1e-10, xtol=1e-14)
print(f"N = 1: calibrate u = {u:.10f},  brentq u = {u_brentq:.10f}")

# (b) prices that the tree cannot give (Step 2): calibrate raises ValueError
for price in [0.60, 0.04]:
    try:
        calibrate(price, spot, strike, maturity, 1, df_annual)
    except ValueError as error:
        print("ValueError:", error)

# (c) picture: tree price against u for N = 1, 2, 5; the calibrated u is where it meets the target
u_grid = np.linspace(1.0, 1.6, 121)
curves, names, u_cal = [], [], []
for steps in [1, 2, 5]:
    DF = df_annual ** (maturity / steps)
    curves.append([balanced_tree_call_price(spot, strike, maturity, steps, df_annual, x)
                   if 1 / DF < x < 2 / DF else np.nan for x in u_grid])
    u_cal.append(calibrate(target, spot, strike, maturity, steps, df_annual))
    names.append(f"tree, N = {steps}  (u = {u_cal[-1]:.4f})")
fig, ax = my_plotter(u_grid, curves,
                     layout={"title": "Calibration: tree price against the up step u",
                             "x_label": "up step u", "y_label": "call price",
                             "hline": target, "vlines": u_cal, "grid": True,
                             "title_fontsize": 13, "label_fontsize": 11, "legend_fontsize": 10,
                             "linewidth": 2, "colors": ["tab:blue", "tab:orange", "tab:green"],
                             "save_path": "figures/q1_calibration.pdf", "show": False},
                     names=names)
```

output (q1_calibrate_test.out)

```
target price (Black-Scholes of Homework 1, sigma = 0.2): 0.105195
N = 1: calibrate u = 1.2214640224,  brentq u = 1.2214640224
ValueError: price 0.600000 not attainable: it must lie in (0.050000, 0.525000)
ValueError: price 0.040000 not attainable: it must lie in (0.050000, 0.525000)
```

(a) The two solvers give the same up step:

$$\begin{aligned} u_{\texttt{calibrate}} &= 1.2214640224 \\ &= u_{\texttt{brentq}} \qquad \checkmark \end{aligned}$$

(b) Step 2 for $N = 1$ prints the tree prices at the two ends of the interval, $V_{\min} = 0.05$ and $V_{\max} = 0.525$ (output); $0.60 \geq V_{\max}$ and $0.04 \leq V_{\min}$, so both raise `ValueError`.

(c) In Figure 6 each curve is flat at $V_{\min} = 0.05$ near $1/DF$, then increasing, and meets the dashed line V^* exactly once, at the dotted line of its calibrated u.

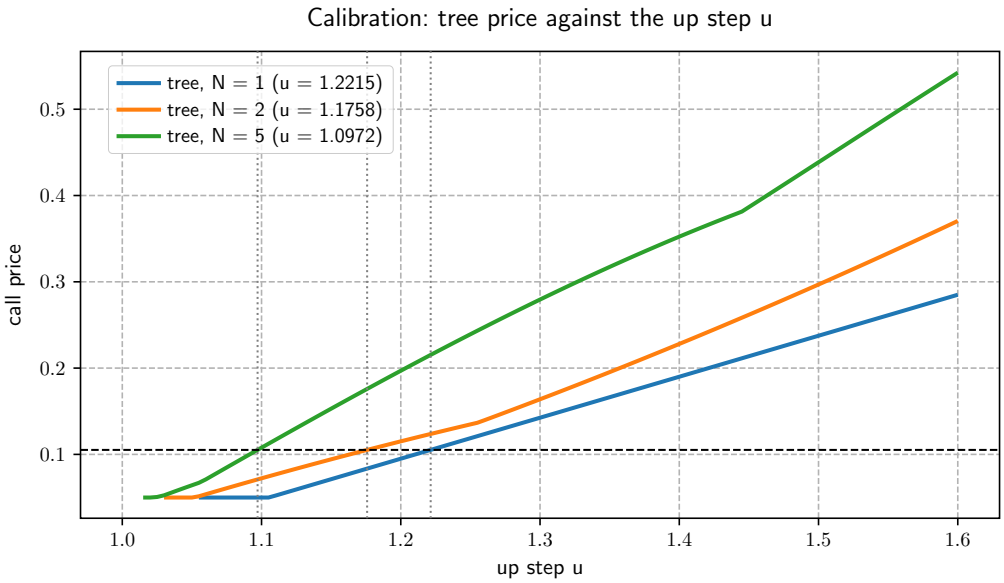


Figure 6: Output (c) of the test ($S_0 = K = 1$, $T = 1$, $DF_1 = 0.95$): tree price $V(u)$ for $N = 1, 2, 5$, the market price $V^* = 0.105195$ (dashed) and the calibrated up steps (dotted).

Q1. `calibrate(price, spot, strike, maturity, steps, df_annual)` (file `calibrator.py`) returns the up step u for which the balanced tree with `steps` steps returns the given call price, with error at most 10^{-10}. Running example ($V^* = 0.105195$): $u = 1.2214640224$ for $N = 1$, equal to the `brentq` solution; $u = 1.1758$ for $N = 2$ and $u = 1.0972$ for $N = 5$ (Figure 6). A price outside $(V_{\min}, V_{\max})$ of Step 2, here (0.05, 0.525), raises `ValueError`.

Q2 — Demo A: the same contract with several numbers of steps.

Contract $S_0 = K = 1$, $T = 1$, $V^* = 0.105195$; for each N the discount factor is scaled, $DF = 0.95^{1/N}$ (Definition 3), so the rate stays flat. After `calibrate` we compute d (Lemma 1), the error, and the tree volatility (Lemma 2, (6)).

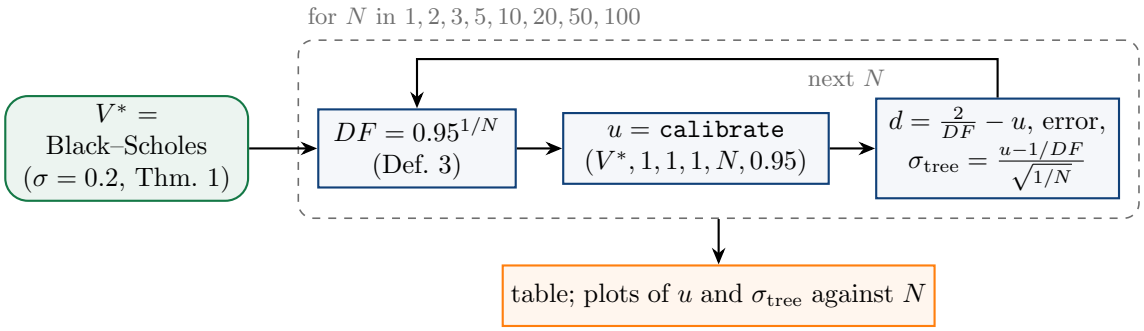


Figure 7: Flowchart of Demo A.

Demo A (demo_A.py)

```
# Demo A: the same contract (S0 = K = 1, T = 1, yearly DF = 0.95) with several numbers of steps
import numpy as np
from plotter import my_plotter
from hw1_call import black_scholes_eur_call
from bg_functions import balanced_tree_call_price
from calibrator import calibrate, tree_vol

spot, strike, maturity, df_annual, sigma = 1.0, 1.0, 1.0, 0.95, 0.2
r = -np.log(df_annual)
target = black_scholes_eur_call(r=r, T=maturity, S0=spot, sigma=sigma, K=strike)[0]
steps_list = [1, 2, 3, 5, 10, 20, 50, 100]
u_list, vol_list = [], []
for steps in steps_list:
    DF = df_annual ** (maturity / steps)                    # scaled DF: flat rate (Def. 3)
    u = calibrate(target, spot, strike, maturity, steps, df_annual)
    error = balanced_tree_call_price(spot, strike, maturity, steps, df_annual, u) - target
    u_list.append(u)
    vol_list.append(tree_vol(u, maturity, steps, df_annual))
    print(f"N = {steps:3d}  DF = {DF:.6f}  u = {u:.6f}  d = {2/DF - u:.6f}  "
          f"tree vol = {vol_list[-1]:.4f}  error = {error:.1e}")

common = {"grid": True, "title_fontsize": 13, "label_fontsize": 11, "legend_fontsize": 10,
          "linewidth": 2, "markers": "o", "logx": True, "show": False}
my_plotter(steps_list, u_list, names=["calibrated u"],
           layout={**common, "title": "Demo A: calibrated up step against N",
                   "x_label": "number of steps N", "y_label": "up step u", "hline": 1.0,
                   "save_path": "figures/demoA_u.pdf"})
my_plotter(steps_list, vol_list, names=["tree volatility (u - 1/DF)/sqrt(dt)"],
           layout={**common, "title": "Demo A: tree volatility against N",
                   "x_label": "number of steps N", "y_label": "tree volatility",
                   "hline": sigma, "save_path": "figures/demoA_vol.pdf"})
```

output (demo_A.out)

```
N =   1  DF = 0.950000  u = 1.221464  d = 0.883799  tree vol = 0.1688  error = -2.8e-17
N =   2  DF = 0.974679  u = 1.175830  d = 0.876127  tree vol = 0.2119  error = -2.8e-17
N =   3  DF = 0.983048  u = 1.126435  d = 0.908054  tree vol = 0.1891  error = -8.3e-17
N =   5  DF = 0.989794  u = 1.097162  d = 0.923461  tree vol = 0.1942  error = 1.5e-16
N =  10  DF = 0.994884  u = 1.067895  d = 0.942390  tree vol = 0.1984  error = 8.3e-17
N =  20  DF = 0.997439  u = 1.046925  d = 0.958211  tree vol = 0.1984  error = 5.0e-16
N =  50  DF = 0.998975  u = 1.029196  d = 0.972857  tree vol = 0.1992  error = -2.2e-16
N = 100  DF = 0.999487  u = 1.020503  d = 0.980523  tree vol = 0.1999  error = 7.1e-16
```

N	$DF = 0.95^{1/N}$	u	$d = 2/DF - u$	$h = u - 1/DF$	$\sigma_{\text{tree}} = h/\sqrt{\Delta t}$	error $V(u) - V^*$
1	0.950000	1.221464	0.883799	0.168832	0.1688	$-2.8 \cdot 10^{-17}$
2	0.974679	1.175830	0.876127	0.149851	0.2119	$-2.8 \cdot 10^{-17}$
3	0.983048	1.126435	0.908054	0.109191	0.1891	$-8.3 \cdot 10^{-17}$
5	0.989794	1.097162	0.923461	0.086851	0.1942	$1.5 \cdot 10^{-16}$
10	0.994884	1.067895	0.942390	0.062753	0.1984	$8.3 \cdot 10^{-17}$
20	0.997439	1.046925	0.958211	0.044357	0.1984	$5.0 \cdot 10^{-16}$
50	0.998975	1.029196	0.972857	0.028169	0.1992	$-2.2 \cdot 10^{-16}$
100	0.999487	1.020503	0.980523	0.019990	0.1999	$7.1 \cdot 10^{-16}$

Table 1: Demo A (from `demo_A.out`; the column h is printed in `outputs.txt`).

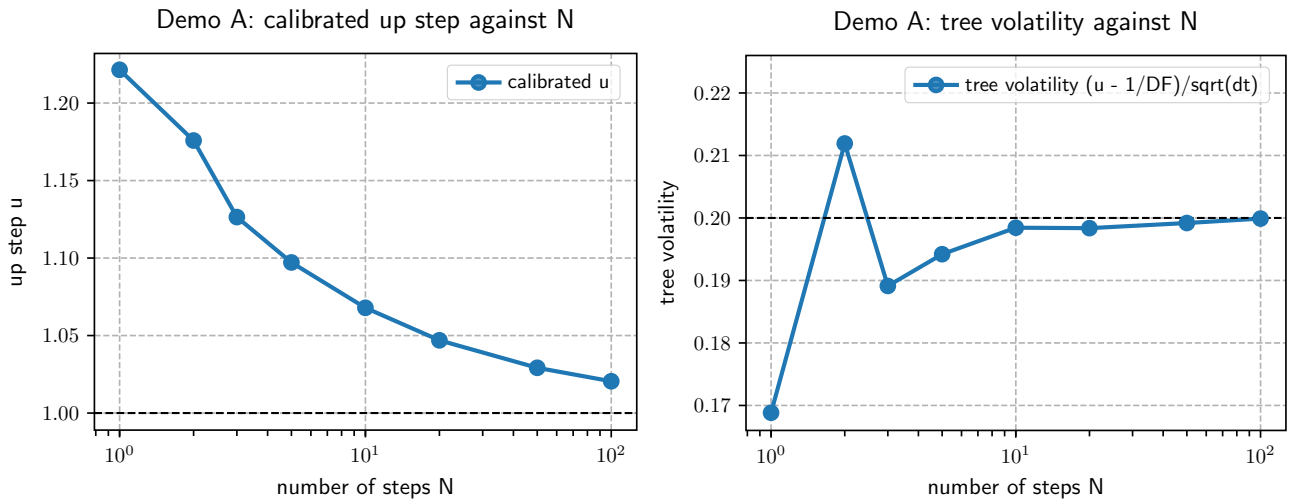


Figure 8: Demo A plots (Q4): calibrated u (left, dashed line $u = 1$) and tree volatility (right, dashed line $\sigma = 0.2$) against N, logarithmic axis.

Reading.

- Every calibration passes Step 5: $|V(u) - V^*| \le 7.1 \cdot 10^{-16}$ in the output.
- u decreases from 1.221464 ($N = 1$) to 1.020503 ($N = 100$), towards 1: $1/DF = 0.95^{-1/N} \to 1$ and the step size $h = u - 1/DF$ goes from 0.168832 to 0.019990 $\approx 0.2\sqrt{1/100}$ (Lemma 2: h is the standard deviation of one step, like $\sigma\sqrt{\Delta t}$).
- The tree volatility tends to 0.2 (0.1999 for $N = 100$), the σ that produced V^*: the calibrated tree recovers the Black–Scholes volatility, as expected from (C1) of part 1. For small N it oscillates (0.1688, 0.2119, 0.1891 for $N = 1, 2, 3$).

Q2 (Demo A). For the same contract, `calibrate` works for every N with the scaled $DF = 0.95^{1/N}$ (Table 1). The calibrated up step decreases to 1, its step size shrinks like $\sqrt{\Delta t}$, and the tree volatility $(u - 1/DF)/\sqrt{\Delta t}$ converges to $\sigma = 0.2$ of the market price (0.1688 for $N = 1$, 0.1999 for $N = 100$; Figure 8).

Q3 — Demo B: one-step tree for several contracts.

$S_0 = 1$, flat rate ($DF_1 = 0.95$, so $DF = 0.95^T$ for one step), $T \in \{0.25, 0.5, 1, 2\}$, $K \in \{0.8, 0.9, 1.0, 1.1, 1.2\}$; each market price is the Black–Scholes price with $\sigma = 0.2$. The code also checks every u against `scipy.optimize.brentq`, another root solver (`assert`). Calibrating a one-step tree to a single European contract is what the earlier edition of the lecture [2] describes.

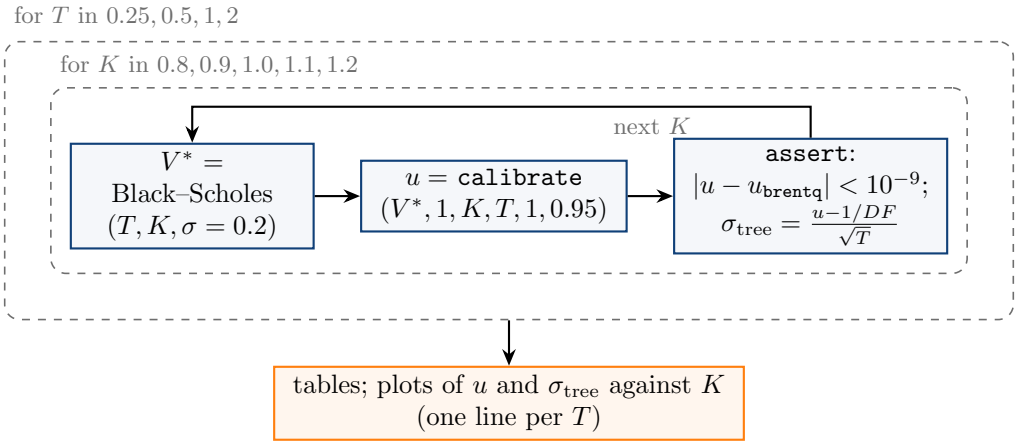


Figure 9: Flowchart of Demo B.

Demo B (demo_B.py)

```python
# Demo B: one-step tree, several contracts (expiries T, strikes K), flat rate (yearly DF = 0.95)
import numpy as np
from scipy.optimize import brentq
from plotter import my_plotter
from hw1_call import black_scholes_eur_call
from bg_functions import balanced_tree_call_price
from calibrator import calibrate, tree_vol

spot, df_annual, sigma = 1.0, 0.95, 0.2
r = -np.log(df_annual)
expiries = [0.25, 0.5, 1.0, 2.0]
strikes = [0.8, 0.9, 1.0, 1.1, 1.2]
u_table, vol_table = [], []
for T in expiries:
    u_row, vol_row = [], []
    for K in strikes:
        price = black_scholes_eur_call(r=r, T=T, S0=spot, sigma=sigma, K=K)[0]
        u = calibrate(price, spot, K, T, 1, df_annual)       # one step: DF = df_annual**T
        DF = df_annual ** T
        u_brentq = brentq(lambda x: balanced_tree_call_price(spot, K, T, 1, df_annual, x)
                          - price, 1 / DF + 1e-10, 2 / DF - 1e-10, xtol=1e-14)
        assert abs(u - u_brentq) < 1e-9                          # another root solver
        u_row.append(u)
        vol_row.append(tree_vol(u, T, 1, df_annual))
    u_table.append(u_row)
    vol_table.append(vol_row)
    print(f"T = {T:4.2f}  u = " + "  ".join(f"{u:.4f}" for u in u_row)
          + "   tree vol = " + "  ".join(f"{v:.3f}" for v in vol_row))

common = {"grid": True, "title_fontsize": 13, "label_fontsize": 11, "legend_fontsize": 10,
          "linewidth": 2, "markers": "o", "hline": None, "show": False}
names = [f"T = {T}" for T in expiries]
my_plotter(strikes, u_table, names=names,
           layout={**common, "title": "Demo B: one-step tree, calibrated up step",
                   "x_label": "strike K", "y_label": "up step u", "legend_loc": "upper center",
                   "save_path": "figures/demoB_u.pdf"})
my_plotter(strikes, vol_table, names=names,
           layout={**common, "title": "Demo B: one-step tree, tree volatility",
                   "x_label": "strike K", "y_label": "tree volatility",
                   "save_path": "figures/demoB_vol.pdf"})
```

output (demo_B.out; per row: u for $K = 0.8, \ldots, 1.2$, then the tree volatilities)

```
T = 0.25  u = 1.2264  1.1369  1.0938  1.1243  1.2041   tree vol = 0.427  0.248  0.162  0.223  0.382
T = 0.50  u = 1.2560  1.1779  1.1421  1.1600  1.2212   tree vol = 0.325  0.215  0.164  0.190  0.276
T = 1.00  u = 1.3195  1.2533  1.2215  1.2282  1.2691   tree vol = 0.267  0.201  0.169  0.176  0.216
T = 2.00  u = 1.4488  1.3916  1.3604  1.3564  1.3778   tree vol = 0.241  0.201  0.178  0.176  0.191
```

The `assert` did not stop the run, so `calibrate` equals the `brentq` solution to 10^{-9} in all 20 cases. Table 2 shows that every market price is attainable (Step 2); Table 3 collects the output.

T		$K = 0.8$	$K = 0.9$	$K = 1.0$	$K = 1.1$	$K = 1.2$
0.25	$V_{\min}$	0.2102	0.1115	0.0127	0.0000	0.0000
	V^*	**0.210466**	**0.116951**	**0.046319**	**0.011978**	**0.002013**
	$V_{\max}$	0.6051	0.5557	0.5064	0.4570	0.4076
0.50	$V_{\min}$	0.2203	0.1228	0.0253	0.0000	0.0000
	V^*	**0.222227**	**0.135441**	**0.069230**	**0.029263**	**0.010316**
	$V_{\max}$	0.6101	0.5614	0.5127	0.4639	0.4152
1.00	$V_{\min}$	0.2400	0.1450	0.0500	0.0000	0.0000
	V^*	**0.246771**	**0.167826**	**0.105195**	**0.060906**	**0.032805**
	$V_{\max}$	0.6200	0.5725	0.5250	0.4775	0.4300
2.00	$V_{\min}$	0.2780	0.1878	0.0975	0.0072	0.0000
	V^*	**0.292778**	**0.221853**	**0.162637**	**0.115717**	**0.080220**
	$V_{\max}$	0.6390	0.5939	0.5487	0.5036	0.4585

Table 2: Demo B: market price V^* (bold) between $V_{\min}$ and $V_{\max}$ of Step 2 (from `outputs.txt`).

	calibrated up step u					tree volatility $\sigma_{\text{tree}} = (u - 1/DF)/\sqrt{T}$				
T	0.8	0.9	1.0	1.1	1.2	0.8	0.9	1.0	1.1	1.2
0.25	1.2264	1.1369	1.0938	1.1243	1.2041	0.427	0.248	0.162	0.223	0.382
0.50	1.2560	1.1779	1.1421	1.1600	1.2212	0.325	0.215	0.164	0.190	0.276
1.00	1.3195	1.2533	1.2215	1.2282	1.2691	0.267	0.201	0.169	0.176	0.216
2.00	1.4488	1.3916	1.3604	1.3564	1.3778	0.241	0.201	0.178	0.176	0.191

Table 3: Demo B: u and σ_{tree} by expiry T (rows) and strike K (columns), from `demo_B.out`.

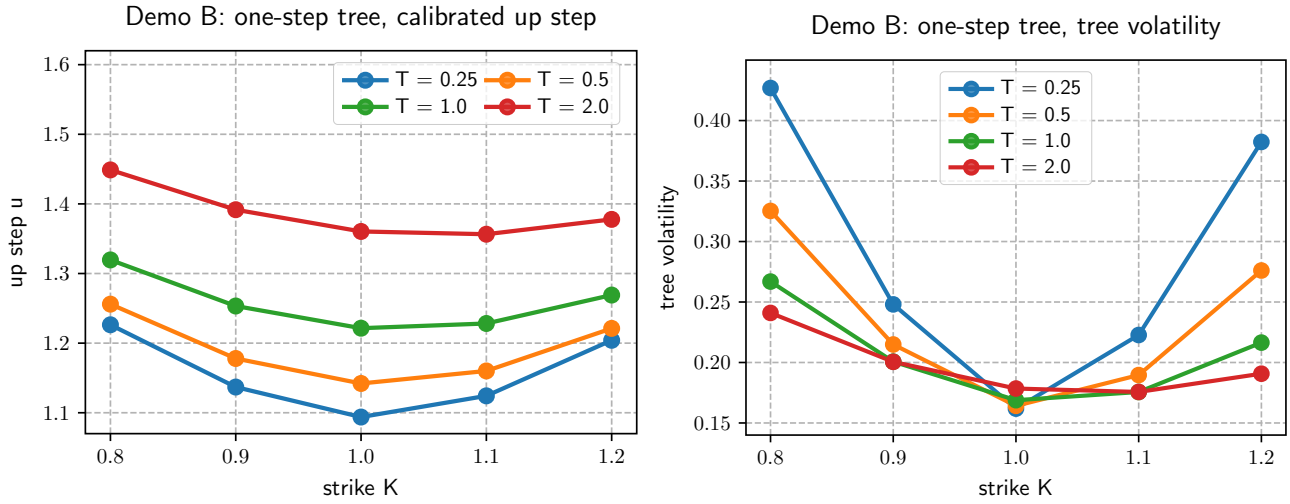


Figure 10: Demo B plots (Q4): calibrated u (left) and tree volatility (right) against the strike, one line per expiry.

Reading.

- Every price is attainable (Table 2); the closest case is $T = 0.25$, $K = 0.8$: $V^* = 0.210466$ against $V_{\min} = 0.2102$.
- For each strike, u grows with T (e.g. $K = 1.0$: 1.0938, 1.1421, 1.2215, 1.3604): one step now lasts T years, and by Lemma 2 $u = 1/DF + \sigma_{\text{tree}}\sqrt{T}$.
- The tree volatility is not flat in K: it is smallest near the money (0.162 at $T = 0.25$, $K = 1.0$; for $T = 2$ at $K = 1.1$, near the mean $S_0/DF = 1/0.95^2 \approx 1.108$ of the leaves, Lemma 2) and larger in the wings (0.427 at $T = 0.25$, $K = 0.8$). A one-step tree has only two leaves and cannot reproduce the Black–Scholes distribution, so each strike needs its own step.

Q3 (Demo B). The one-step tree can be calibrated to all 20 contracts (flat rate, $DF = 0.95^T$), and `calibrate` agrees with `brentq` to 10^{-9}. The calibrated u grows with the expiry (Table 3); the tree volatility depends on the strike, smallest near the money and larger in the wings (Figure 10): one step cannot match Black–Scholes for all strikes with one u.

Q4 — the calibrated trees.

To see how the calibrated model works, we draw the trees of the lecture code for the contract of Demo A ($S_0 = K = 1$, $T = 1$, $DF_1 = 0.95$, $V^* = 0.105195$): the spot tree (cell 5) and the price tree (cell 8) with the calibrated u. $N = 2$ and $N = 5$: every node; $N = 50$: all 1326 nodes, labels on the first, middle and last level (bottom, middle and top node). V is the value returned by `create_discounted_price_tree`: discounted to $t = 0$ (Definition 1, cell 8), so the root is the price.

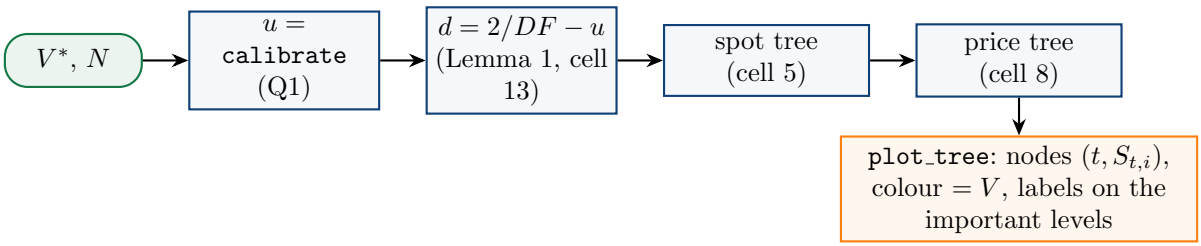


Figure 11: Flowchart of Q4 (file `tree_plot.py`).

new code (tree_plot.py)

```python
"""Homework 2, part 2 (Q4): pictures of the calibrated balanced trees.

calibrated_trees(...) calibrates u with calibrate (Q1) and builds the two trees of the lecture
code: the spot tree (create_spot_tree) and the price tree (create_discounted_price_tree).
plot_tree(...) draws them: x = time step t, y = spot S_{t,i}, colour = value of the node as
returned by create_discounted_price_tree (discounted to t = 0; the root is the price).
Small trees: every node is labelled. Large trees: all nodes are drawn, labels only on the
important levels (first, second, middle, last) and on the top, middle and bottom node.
"""
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.collections import LineCollection

from calibrator import calibrate
from prof_tree import calcBalancedDownStep, create_discounted_price_tree, create_spot_tree


def calibrated_trees(price: float, spot: float, strike: float, maturity: float, steps: int,
                     df_annual: float):
    """Return (u, d, DF, spot_tree, price_tree) of the balanced tree calibrated to price."""
    u = calibrate(price, spot, strike, maturity, steps, df_annual)
    discount_factor = df_annual ** (maturity / steps)              # flat rate (Definition 3)
    d = calcBalancedDownStep(u, discount_factor)                   # lecture, cell 13
    spot_tree = create_spot_tree(spot, u, d, steps)                # lecture, cell 5
    price_tree = create_discounted_price_tree(spot_tree, discount_factor, strike)   # cell 8
    return u, d, discount_factor, spot_tree, price_tree


def important_nodes(level: int) -> list[int]:
    """Indices i of the nodes of level t to label: all of them if few, else bottom/middle/top."""
    return list(range(level + 1)) if level <= 5 else sorted({0, level // 2, level})


def plot_tree(spot_tree, price_tree, strike=None, label_levels=None, title=None, ax=None,
              fontsize=8, logy=None):
    """Draw a spot tree with its price tree. Returns (fig, ax)."""
    steps = len(spot_tree) - 1
    if label_levels is None:
        label_levels = range(steps + 1) if steps <= 5 else sorted({0, steps // 2, steps})
    if logy is None:
        logy = steps > 5                       # log scale: the large tree looks symmetric
    if ax is None:
        ax = plt.subplots(figsize=(9, 5))[1]
    fig = ax.figure
    # edges: node (t, i) -> down child (t+1, i) and up child (t+1, i+1)
    segments = [[(t, spot_tree[t][i]), (t + 1, spot_tree[t + 1][j])]
                for t in range(steps) for i in range(t + 1) for j in (i, i + 1)]
    ax.add_collection(LineCollection(segments, colors="0.75", linewidths=0.6, zorder=1))
    # nodes, coloured by the value of the price tree
    xs = [t for t in range(steps + 1) for _ in spot_tree[t]]
    ys = [s for level in spot_tree for s in level]
    vs = [v for level in price_tree for v in level]
    size = 60 if steps <= 5 else max(4, 400 / steps)
    points = ax.scatter(xs, ys, c=vs, cmap="viridis", s=size, zorder=2)
    fig.colorbar(points, ax=ax, label="node value (discounted to t = 0)")
    for t in label_levels:
        for i in important_nodes(t):
            ax.annotate(f"S={spot_tree[t][i]:.3f}\nV={price_tree[t][i]:.4f}",
                        (t, spot_tree[t][i]), textcoords="offset points", xytext=(4, 4),
                        fontsize=fontsize, zorder=3,
                        bbox={"boxstyle": "round,pad=0.15", "fc": "white", "alpha": 0.8, "lw": 0})
    if strike is not None:
        ax.axhline(strike, linestyle="--", color="tab:red", linewidth=1, label=f"strike K = {strike}")
        ax.legend(loc="upper left", fontsize=fontsize + 1)
    if logy:
        ax.set_yscale("log")
        ticks = [2.0 ** k for k in range(-6, 7) if min(ys) <= 2.0 ** k <= max(ys)]
        ax.set_yticks(ticks, [f"{x:g}" for x in ticks])
        ax.minorticks_off()
    ax.set_xlabel("time step t")
    ax.set_ylabel("spot S" + (" (log scale)" if logy else ""))
    ax.set_xlim(-0.3, steps + 0.8 + 0.08 * steps)
    if title:
        ax.set_title(title)
    return fig, ax
```

run (trees_demo.py)

```
# Q4: the calibrated balanced trees of Demo A (S0 = K = 1, T = 1, yearly DF = 0.95), N = 2, 5, 50
import numpy as np
from hw1_call import black_scholes_eur_call
from tree_plot import calibrated_trees, important_nodes, plot_tree

spot, strike, maturity, df_annual = 1.0, 1.0, 1.0, 0.95
r = -np.log(df_annual)
target = black_scholes_eur_call(r=r, T=maturity, S0=spot, sigma=0.2, K=strike)[0]
print(f"target price: {target:.6f}")

for steps in [2, 5, 50]:
    u, d, DF, spot_tree, price_tree = calibrated_trees(target, spot, strike, maturity, steps,
                                                       df_annual)
    print(f"\nN = {steps}: u = {u:.6f}, d = {d:.6f}, DF = {DF:.6f}, "
          f"root value = {price_tree[0][0]:.6f}")
    for t in sorted({0, 1, steps // 2, steps}):
        nodes = [f"(S={spot_tree[t][i]:.4f}, V={price_tree[t][i]:.6f})" for i in important_nodes(t)]
        for k in range(0, len(nodes), 3):                    # at most 3 nodes per line
            print(f"  t = {t:2d}: " + "  ".join(nodes[k:k + 3]))
    # backward induction check at the root: V_0 = (V_up + V_down) / 2 (p_up = p_down = 1/2)
    print(f"  (V_up + V_down)/2 at t = 1: {(price_tree[1][0] + price_tree[1][1]) / 2:.6f}")
    plot_tree(spot_tree, price_tree, strike=strike,
              title=f"Calibrated balanced tree, N = {steps} (u = {u:.4f})")[0].savefig(
        f"figures/tree_N{steps}.pdf")
```

output (trees_demo.out)

```
target price: 0.105195

N = 2: u = 1.175830, d = 0.876127, DF = 0.974679, root value = 0.105195
  t =  0: (S=1.0000, V=0.105195)
  t =  1: (S=0.8761, V=0.014334)  (S=1.1758, V=0.196057)
  t =  2: (S=0.7676, V=0.000000)  (S=1.0302, V=0.028667)  (S=1.3826, V=0.363447)
  (V_up + V_down)/2 at t = 1: 0.105195

N = 5: u = 1.097162, d = 0.923461, DF = 0.989794, root value = 0.105195
  t =  0: (S=1.0000, V=0.105195)
  t =  1: (S=0.9235, V=0.050070)  (S=1.0972, V=0.160320)
  t =  2: (S=0.8528, V=0.014997)  (S=1.0132, V=0.085144)  (S=1.2038, V=0.235497)
  t =  5: (S=0.6716, V=0.000000)  (S=0.7979, V=0.000000)  (S=0.9480, V=0.000000)
  t =  5: (S=1.1263, V=0.119973)  (S=1.3381, V=0.321234)  (S=1.5898, V=0.560351)
  (V_up + V_down)/2 at t = 1: 0.105195

N = 50: u = 1.029196, d = 0.972857, DF = 0.998975, root value = 0.105195
  t =  0: (S=1.0000, V=0.105195)
  t =  1: (S=0.9729, V=0.087210)  (S=1.0292, V=0.123180)
  t = 25: (S=0.5026, V=0.000000)  (S=0.9877, V=0.060739)  (S=2.0533, V=1.051299)
  t = 50: (S=0.2526, V=0.000000)  (S=1.0320, V=0.030390)  (S=4.2160, V=3.055198)
  (V_up + V_down)/2 at t = 1: 0.105195
```

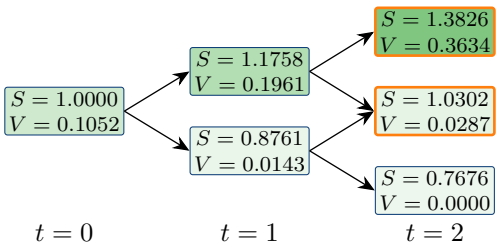


Figure 12: Calibrated tree, $N = 2$ ($u = 1.175830$, $d = 0.876127$, $DF = 0.974679$). Up moves go up. Colour: value V; orange border: leaf above K.

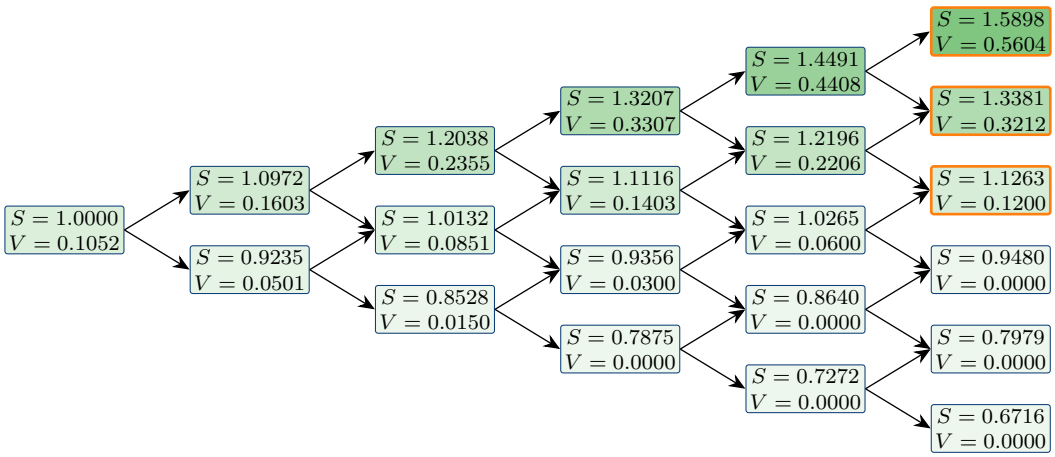


Figure 13: Calibrated tree, $N = 5$ ($u = 1.097162$, $d = 0.923461$, $DF = 0.989794$); from $t = 0$ (left) to $t = 5$ (right).

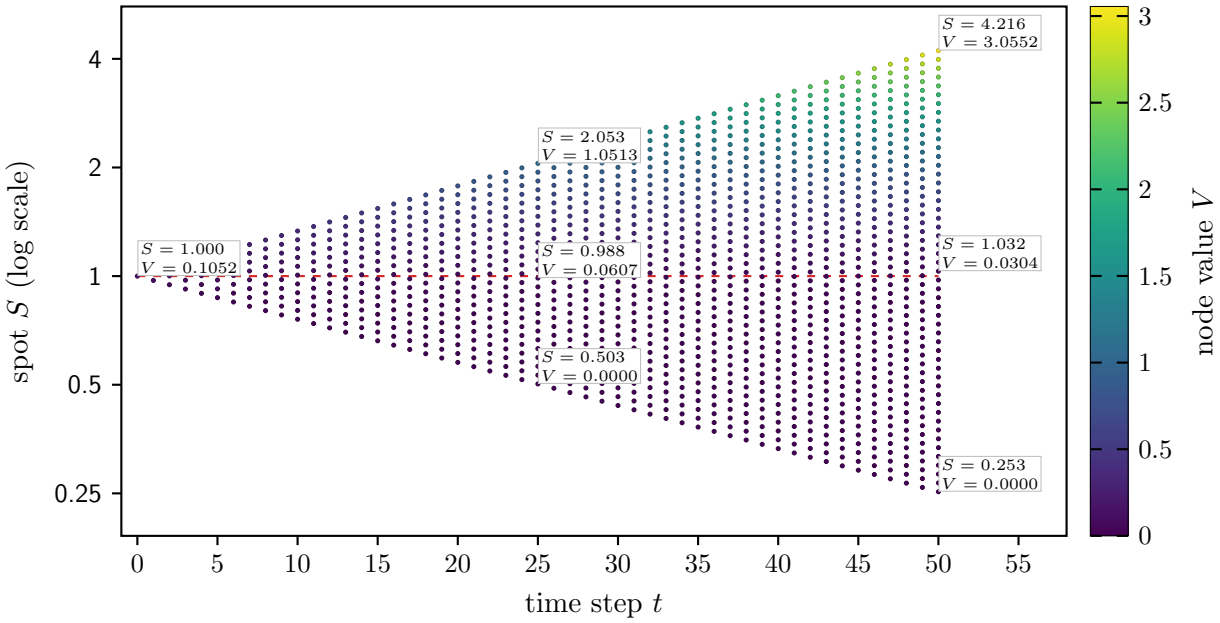


Figure 14: Calibrated tree, $N = 50$ ($u = 1.029196$): all nodes, colour $= V$; red dashed line: strike $K = 1$.

Reading.

- *Root.* For $N = 2, 5, 50$ the root value is $0.105195 = V^*$: the calibration of Q1 at work.
- *Backward step.* Each node is the average of its two children ($\tilde{p} = \tilde{q} = \frac{1}{2}$, Definition 2; the price tree is discounted to $t = 0$, so no factor is applied inside the tree). $N = 2$, root:

$$\tfrac{1}{2}\,(0.014334 + 0.196057) = 0.105195. \qquad \checkmark$$

- *Leaves.* A leaf is the payoff times $DF^N = DF_1^T = 0.95$ (cell 8; Definition 3): $N = 5$, $S = 1.126288$:

$$0.95\,(1.126288 - 1) = 0.119973. \qquad \checkmark$$

  Leaves below K are worth 0; only the orange leaves (Figures 12, 13) feed the price.

- *Shape.* With more steps the up step shrinks (1.1758, 1.0972, 1.0292), but the leaves spread more (0.253 to 4.216 for $N = 50$); in log scale the tree is a symmetric fan around the growth of the money market (Lemma 2).

Q4 (trees). The pictures show the calibrated model at work: the leaves carry the discounted payoff, each node is the average of its two children, and the root is the given price 0.105195 for every N.

Conclusions.

- `calibrate` (Q1) returns the up step of the balanced tree with the given call price: Steps 1–5 of Figure 2, error at most 10^{-10} (in Demo A at most $7.1 \cdot 10^{-16}$).
- Only prices between the tree prices at the two ends of the interval of u can be calibrated (Step 2); other prices raise `ValueError`.
- Every calibrated u agrees with a second root solver (`brentq`) to 10^{-9} (Q1, Q3).
- Demo A: with more steps the calibrated tree volatility tends to $\sigma = 0.2$ of the market price (Q2).
- Demo B: a one-step tree needs a different step for each strike and expiry (Q3).
- Trees (Q4): in every calibrated tree the root is the given price, built backward from the discounted payoffs at the leaves.

Comment (to keep in mind).

- *Why the bracket of Step 3.* The first tries applied `minimize` (L-BFGS-B, Nelder–Mead) on all of $(1/DF, 2/DF)$: they can stop in the flat part, where the error is constant and the gradient is zero, and L-BFGS-B (finite-difference gradient) also stopped at errors of order 10^{-9}. Inside the bracket this cannot happen. A root solver on $V(u) - V^*$ in the bracket (e.g. `scipy.optimize.brentq`) is an alternative.
- *Lecture code.* `create_discounted_price_tree` reads the up step as `spot_tree[1][-1]` $= uS_0$, which is u only for $S_0 = 1$ (all runs here use $S_0 = 1$); and it multiplies $\tilde{p}$ with the down child (index i counts up moves), which does not matter here because the tree is balanced, $\tilde{p} = \tilde{q} = \frac{1}{2}$.

References.

[1] Lecture 4 of this course: `Tree.ipynb` and `Binomial_tree_coding.ipynb`, repository QuantCourseBP, folder `Lectures/2026_fall_BME/04_Tree`.

[2] Earlier edition of the same tree lecture: `Tree.ipynb`, repository QuantCourseBP, folder `Lectures/2023_fall_ELTE/07_Tree` (section "Basic concept of binomial tree model (cont.)").

[3] S. E. Shreve, *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*, Springer, 2004 (the literature of the lecture, `Tree.ipynb` cell 58), Chapter 1.

[4] Homework 1 (Lesson 3) and Homework 2, part 1 (Background) of this course.